Richfield Hybrid Learning

IT PROJECT 700

SYSTEM DESIGN 2026

A Web-Based Mock Test Application for the NaTIS Computerised Learner's Licence Test (CLLT)

Miriam Phily
Jason Jenkinson
Dirk Potgieter
Sergio Daniels
Shenevra Govender
Josh Dekiewit

30/10/2026

Table of content

Table of content **2**
4.1 Introduction **4**
4.2 System Design **5**
4.2.1 General System Operation 5
4.2.2 Learner Testing Process 6
4.2.3 Results and Test History 6
4.2.4 Question Bank Management 7
4.2.5 System Security and Data Management 7
4.2.6 Overall System Design Objective 8
4.3 Architectural Design **9**
Overview 9
Back-End Layers 10
Components 11
API Design 12
How a Test Flows Through the System 14
Sessions and Guest Access 14
Business Rules in the Code 14
Security 16
How the Architecture Supports the Data Model 17
Running the Back End 18
Limitations and Future Improvements 18
4.4 Physical Design **19**
4.4.1 Development Environment 19
4.4.2 Source Code and Hosting 19
4.4.3 System Components and Connections 20
4.4.4 Supported Devices and Performance 20
4.4.5 Communication and Documentation 21
4.5 Database Design **22**
Simulated user flow based off database design 24
Proposed SQL for our Database 25
4.6 Program Design (Program Pseudo code) 32
Framework and technologies 32
Project structure 33
4.6.3 Admin key check 39
4.6.4 Bulk question import 40
4.6.5 Input and output data 41
4.7 Interface Design **42**
Attached Resources: 42
4.8 Security and Backup Design 45

4.8.1 Password Protection and Login Sessions 45
4.8.2 Access to Results and Question Management 45
4.8.3 Checking Inputs and Test Results 46
4.8.4 Security of the Hosted Application 47
4.8.5 Backup Schedule and Storage 47
4.8.6 Restoring Data and Checking Backups 48

4.1 Introduction

In the System Design Phase we transition from defining what the application must achieve to engineering how to in fact build this system.
We build on the functional and non functional requirements and the database schemes.
This section outlines the architectural, technical, physical and interface blueprints that we will use for the NaTIS Computerised Learner's Licence Test (CLLT) mock test application.

Our primary objective of this phase is to create a concrete structure of the web app while maintaining the user's needs. Namely free access, informative results and a robust question bank with randomization that mimics the CLLT.
As we are a six person team developing this app, we need modularity in design, and data persistence.

To achieve these goals, the system design leverages a tried and tested technology stack.
Our Frontend is composed of HTML5, CSS, and vanilla JavaScript to ensure lightweight, responsive performance across desktop and mobile devices.
Our Backend is composed of Java, utilizing an object-oriented Model-View-Controller (MVC) architectural pattern to separate business logic, data access, and user interfaces.
Our Database will be MySQL, a relational database managed by JDBC to securely store our user accounts test history and verified question bank.

4.2 System Design

Our proposed system is a free, web-based mock examination application intended to help South African learners prepare for the NaTIS Computerised Learner's Licence Test (CLLT). It will provide learners with an opportunity to practise in an environment similar to the actual examination, while also giving them feedback on their performance and the areas they need to improve.

The application will be accessible through a web browser on desktop computers, tablets and mobile devices. Users will not need to install any additional software to access it. From the landing page, learners will be able to start a mock test without creating an account or logging in. Those who choose to register will have access to additional features, such as viewing their previous test attempts and tracking their progress.

4.2.1 General System Operation

The application will consist of three main components: the frontend, backend and database. Each component will have its own responsibilities, but they will work together to provide the functionality of the system.

The frontend will be what learners interact with when using the application. It will display the different pages, questions, answer options, navigation controls and test results. The interface will be kept relatively simple so that users can navigate the application without difficulty. It will also be designed to work across different screen sizes.

The backend will handle the main operations of the application. This includes retrieving questions from the database, selecting questions for each test, processing submitted answers and calculating the learner's results. It will also handle user sessions and ensure that features such as test history and question management are only accessible where appropriate.

The database will store the information needed for the application to function. This includes user accounts, question categories, questions, answer options and records of completed test attempts. These records will be linked so that the system can retrieve the correct information when needed and keep track of each learner's results.

4.2.2 Learner Testing Process

The main purpose of the application is to allow learners to complete mock examinations that resemble the actual CLLT. When a learner starts a test, the system will select questions from the available question bank and randomise their selection.

The questions will be organised into three main categories: Rules of the Road, Road Signs and Controls of the Vehicle. Only questions that have been verified and marked as active will be included in the selection process. This will help ensure that learners are practising with relevant and reliable questions.

During a test, learners will be shown one question at a time along with the available answer options. They will select their answer and move through the examination using the navigation controls provided. Their progress and selected answers will be tracked until they submit the test.

Once the examination has been submitted, the system will check the learner's answers against the correct answers stored in the database. It will then calculate their overall mark and determine how well they performed in each of the three categories.

4.2.3 Results and Test History

After completing a test, learners will be presented with a results page showing their overall score and whether they passed or failed based on the examination requirements.

The results will also provide a breakdown of their performance across the different categories. Learners will be able to see which questions they answered incorrectly and review the correct answers. Where explanations are available, these will also be displayed to help learners understand their mistakes.

Registered learners will be able to access their previous test attempts. This will give them a way to compare their results and see whether they are improving over time. Learners who use the application as guests will still be able to complete tests, but their access to previous results will be limited to their current session.

Providing this level of feedback is important because simply showing a learner their final mark does not necessarily tell them what they need to work on. By showing their performance in each category, the application will make it easier for them to identify weaker areas and focus their studies accordingly.

4.2.4 Question Bank Management

The system will also include an administrative section for managing the question bank. Access to this section will be restricted to authorised administrators, who will be able to add new questions, update existing ones and change their status.

Each question will be linked to a category and will contain its question text, available answer options, correct answer and relevant source information. A verification status will also be maintained so that questions can be checked before being made available for testing.

If a question is found to be incorrect or outdated, it can be marked as inactive rather than permanently removed from the database. This is useful because it prevents changes to the question bank from affecting records of previous test attempts.

Having these management features will also make it easier to maintain and expand the question bank as more questions are sourced and verified.

4.2.5 System Security and Data Management

Security will need to be considered when handling user accounts and examination records. Passwords will be stored using secure hashing rather than as plain text, and parameterised database queries will be used to help prevent SQL injection.

The backend will also check whether users are authorised to access certain information. For example, registered learners should only be able to view their own test history and results. Guest users will be limited to the test attempts associated with their current session, while administrative functions will only be available to authorised administrators.

The database will also be structured to maintain the relationships between its records. For example, answer options must be linked to their respective questions, while questions must belong to valid categories. Completed test attempts should retain their original results even if a question is later updated or made inactive.

These measures will help keep the information stored by the application organised and reduce the possibility of unauthorised access or inconsistencies in the results.

4.2.6 Overall System Design Objective

The main objective of the system design is to provide learners with an accessible and straightforward way to prepare for the NaTIS Computerised Learner's Licence Test. The application will bring together a large, organised question bank, randomised mock examinations, automatic marking and detailed results to provide a more useful testing experience.

The system will be divided into separate components, allowing the frontend, backend and database to be developed and maintained independently. This will also make it easier for the six group members to work on different parts of the application during development.

This design will serve as the basis for the more detailed architectural, physical, database, program, interface and security designs covered in the following sections.

4.3 Architectural Design

Learner's Licence Mock Test: back-end architecture, based on the Spring Boot code in the Backend folder.

Overview

The system is a three-tier client-server application. The front end runs in the learner's browser and talks to the back end only through a REST API that sends and receives JSON. The back end holds all the business rules (picking questions, marking, pass marks) and is the only part that touches the database. The database stores the six tables from the data model in section 3.8.

Tier	Technology	Responsibility
Frontend	Vanilla HTML, CSS and JavaScript (the Views folder, designed in Figma)	Shows the start page, test questions and results, and calls the API
Backend	Java 17+, Spring Boot (Spring Web, Spring Data JPA), built with Maven	REST API, session handling, test generation, marking, question bank management
Database	H2, file-based (Backend/data/), accessed through JPA and Hibernate	Stores users, categories, questions, answer options, attempts and attempt questions

H2 was chosen because it needs no separate installation, so every group member can run the project with one command. Because the code talks to the database through JPA, moving to a server database such as PostgreSQL later only means changing the datasource settings in application.properties and adding the driver.

Back-End Layers

The back end is split into layers. Each layer only talks to the one below it, so a change in one place (for example a new pass mark) does not ripple through the rest of the code.

Layer	Package	What it does
Controller	com.mocktest.controller	Receives HTTP requests, checks who is asking, calls a service and returns JSON. Contains no business rules
Service	com.mocktest.service	Business logic: test generation, answer validation, marking and question bank rules
Repository	com.mocktest.repo	Database access through Spring Data JPA interfaces
Model	com.mocktest.model	JPA entities, one per table in the data model
DTO	com.mocktest.dto	Plain data objects that define exactly what goes in and out of the API, so entities are never sent directly

Configuration	com.mocktest.config	CORS settings and the starter data loader

Components

Component	Layer	Responsibility
AuthController	Controller	Register, log in, log out and "who am I"
TestController	Controller	Start a test, save answers, submit, view results and history. Makes sure learners can only open their own attempts
AdminController	Controller	Question bank management, protected by an admin key
TestService	Service	Randomly selects questions per category, validates answers, marks the test and saves the score and result
QuestionService	Service	Adds, verifies, rejects, retires and corrects questions, and rejects duplicates

PasswordHasher	Service	Hashes and checks passwords with PBKDF2
UserRepository, CategoryRepository, QuestionRepository, AnswerOptionRepository, TestAttemptRepository, AttemptQuestionRepository	Repository	One per table, plus the queries the services need
User, Category, Question, AnswerOption, TestAttempt, AttemptQuestion	Model	Entities that map to the tables in section 3.8
CorsConfig	Configuration	Lets the browser front end call the AP
DatabaseSeeder	Configuration	Loads three sample categories and questions on first start

API Design

The API is designed around the things that the learner can do, all paths start with /api

Method	Path	Purpose
POST	/auth/register, /auth/login,	Account Handling

	/auth/logout	
GET	/auth/me	Current user or a guest
GET	/catergories	List the topic areas
POST	/tests	Start a new mock test and return its questions with shuffled options
PUT	/tests/{id}/answers	Save or clear the option chosen for one question
POST	/tests/{id}/submit	Mark the test and save the score and pass or fail
GET	/tests/{id}/results	Overall score, category breakdown and incorrect questions
GET	/history	A logged-in learner's earlier attempts
GET,POST	/admin/questions	List questions or add a new one
POST	/admin/questions/{id}/verify, /retire, /correct	Verify or reject, retire, or replace a question

Errors use standard HTTP status codes: 400 for invalid input, 401 when a login is needed, 403 when the attempt is not yours or the admin key is wrong, 404 when something does not exist and 409 for conflicts such as submitting a test twice or adding a duplicate question.

How a Test Flows Through the System

1. The learner asks to start a test (POST /tests). TestController passes the logged-in user ID, or none for a guest, to TestService.
2. TestService creates a TestAttempt. For every category it takes the Verified and active questions, shuffles them and picks the number required for that category.
3. Each chosen question is saved as an AttemptQuestion row with its position, so the system always knows what was asked. The questions are returned with their options in random order, and the correct answer is never sent.
4. As the learner answers, each choice is sent to PUT /tests/{id}/answers. The service checks that the attempt is not already submitted and that the option belongs to that question, then saves it.
5. On submit, the service compares each selected option with the correct one, totals the marks overall and per category, applies the required mark for each category and saves the score and pass or fail on the attempt.
6. The results and the list of incorrect questions, with explanations, are returned to the front end. Later requests for the same results read the saved score, so an old result never changes.

Sessions and Guest Access

The back end uses the server's HTTP session, which is tracked with a cookie. When a learner logs in, their user ID is stored in the session. A guest has no user ID, so the attempt is saved with an empty user and the attempt ID is remembered in that guest's session. Before any attempt is read or changed, TestController checks that it belongs to the logged-in user, or to the guest's session. Another person cannot open someone else's attempt. Because the session depends on a cookie, the front end has to send its requests with credentials: "include".

Business Rules in the Code

Rule	Where it is enforced

Only questions that are both Verified and active are used in new tests	QuestionRepository.findEligible
A question needs a source, at least two options and exactly one correct option	QuestionService.create
Duplicate question text is rejected	QuestionService.create
New questions start as Unverified	QuestionService.create
A corrected question is added as a new record and the old one is made inactive, so earlier tests still show what the learner saw	QuestionService.correct
A selected option must belong to the same question	TestService.answer
A submitted test cannot be changed or submitted again	TestService.submit and TestService.answer
Pass or fail takes the required mark for each category into account	TestService.compute and using the RULES table
Score and pass or fail are stored on the attempt	TestService.submit

The number of questions and the required mark for each category are kept in one place, the RULES map in TestService, so they can be changed without touching the rest of the code. If a category has fewer questions in the bank than the test asks for, the required mark is scaled down to match.

Security

Concerns	How its handled
Passwords	Hashed with PBKDF2 (HMAC-SHA256, 120,000 iterations) and a random salt per user. The readable password is never saved
Access to attempts	Checked on every request against the session
Administrator functions	Require an X-Admin-Key header that matches the app.admin-key setting
Cross-origin requests	CorsConfig allows browser calls to /api/**

How the Architecture Supports the Data Model

The processes in the Level 1 data flow diagram in section 3.8 each have a home in the code.

DFD processes	Implemented by
1.0 Manage Accounts and Sessions	AuthController, PasswordHasher, the HTTP session
2.0 Generate Mock Test	TestService.start
3.0 Record Answers	TestService.answer
4.0 Mark Test and Calculate Results	TestService.submit and TestService.compute
5.0 Display Results and History	TestService.results, TestService.history, TestController
6.0 Maintain Question Bank	AdminController, QuestionService

The six entities in the model package match the six tables in the ERD, with the same keys and relationships. Hibernate creates and updates the tables from these entities when the application starts.

Running the Back End

```
PS C:\Users\user\Learners-License-Mock-Test> cd Backend
PS C:\Users\user\Learners-License-Mock-Test\Backend> .\mvnw.cmd spring-boot:run
```

The API is then available at http://localhost:8080. The H2 database file is created in Backend/data/, which is excluded from Git.

Limitations and Future Improvements

- The administrator is protected by one shared key. A full version would use administrator accounts and roles.
- CORS currently allows any origin for development. It should be limited to the real front-end address before deployment.
- The H2 file database suits development and demonstration. A server database would be better for many users at once.
- The pass marks in RULES should be checked against the official test requirements.
- The sample questions should be replaced with real, sourced and verified questions through the admin endpoints.
- Automated tests for the services (marking and test generation in particular) have not been written yet.

4.4 Physical Design

4.4.1 Development Environment

We will each use our own computer to work on our part of the project. Members doing the backend will need Java and an editor or development environment. Members doing the frontend will need a code editor and a browser to check their pages.

The current backend documentation specifies Java 17 and includes the Maven wrapper. We will use the wrapper to build the project so that members can follow the same build process on their computers.

For the frontend, we will check the pages in the browser while working on them. For instance, a page might display correctly on a computer but have buttons that are too small on a phone. We will need to check these problems as we develop the pages and again when the frontend is connected to the backend.

The backend documentation currently describes an H2 database in the Backend/data folder. This allows members to run the application locally without having to set up a separate database server first.

Our previous phases specified MySQL, so our planned setup is to use H2 during local development and MySQL for deployment. Before moving to MySQL, we will need to transfer the required data and check that the application still works. This will include registration, saving answers and opening previous test results.

4.4.2 Source Code and Hosting

GitHub is where we share the coding that has been done. Members can upload their changes there so that the rest of the group can access the updated project. It also keeps a history of the changes, which will help if we need to check what was changed or return to an earlier version.

Members will work on separate branches and have their changes checked before they are merged. This follows the requirement in Section 3.7 that another group member must review the work. Before we demonstrate the project, we will need to check that we have a working version with the different parts connected.

We also need somewhere to run the application online. GitHub stores our code, and GitHub Pages can host the HTML, CSS and JavaScript frontend. However, the complete application also needs a server that can run Java and keep the database available (GitHub, n.d.).

Our proposed setup is to have the frontend and Java backend available through the same website address. The backend will then connect to MySQL. This should make it easier to manage the connection between the pages and the backend, including login sessions.

We still need to confirm the hosting provider. Free cloud hosting was considered in Phase 2, but we need to check that the service can run our application within its limits. It must support Java, HTTPS and storage that keeps the database and question images after a restart.

The hosting must stay within the no-cost requirement of the project. We do not need to buy a domain for submission. If the online setup is not ready for the demonstration, we can run the application on a group member's computer to show how it works. We would still need to finish the online deployment afterwards.

4.4.3 System Components and Connections

The learner will open the application in their browser. Their device will display the questions and answer options, and the browser will send their selections to the backend.

The Java backend will handle the processing. For instance, when a learner submits a test, Java will check their answers and calculate the results. It will also manage sessions, select questions and save the learner's progress.

The database will hold the records identified in Section 3.8. These are Users, Categories, Questions, Answer Options, Test Attempts and Attempt Questions. The browser will get information through the backend, which will control access to these records.

We also need to keep the question images available. Some questions will use pictures of road signs, so the application must still be able to find those pictures after an update or restart. We will keep the images in persistent storage and record their locations in the database. The backups will include the database and its matching images.

The connection will follow this order:

Learner's browser → website and Java backend → MySQL database

The public website will use HTTPS. Database connection details will remain on the server, and database access will be restricted to the backend. Learners will need an internet connection to use the hosted application.

4.4.4 Supported Devices and Performance

We want learners to use the application on the devices they already have. This includes phones and older computers that do not have much processing power. They will use their browser without needing to install Java, database software or a separate application.

As stated in Section 3.7, we will check the application on screen widths from 360 pixels upwards and test it in Chrome, Firefox, Edge and Safari. The questions must be readable on a phone, and learners must be able to select answers without struggling with small buttons. Question text will be at least 16 pixels, with enough space between the controls.

We will keep the pages small by using plain HTML, CSS and JavaScript and avoiding unnecessary animations and large libraries. We will also compress images while keeping the road signs clear enough to identify. This should help learners who are using mobile data.

The performance targets from Phase 3 will be checked during testing. The login and test pages should load within three seconds on a 4G connection. Moving between questions already loaded for an attempt should take less than one second. Starting a test and displaying results after submission should each take no more than three seconds.

We will also need to check that the database can hold at least 1,200 questions with their related records. The selected hosting setup must be tested with 50 learners taking tests at the same time. We will need the test results before we can say that these requirements have been met.

4.4.5 Communication and Documentation

We use WhatsApp to chat and inform each other of our progress. We also use it to plan and check whether we are on time with our part of the project. If someone needs help or we are waiting for work to be uploaded, we can discuss it in the group chat.

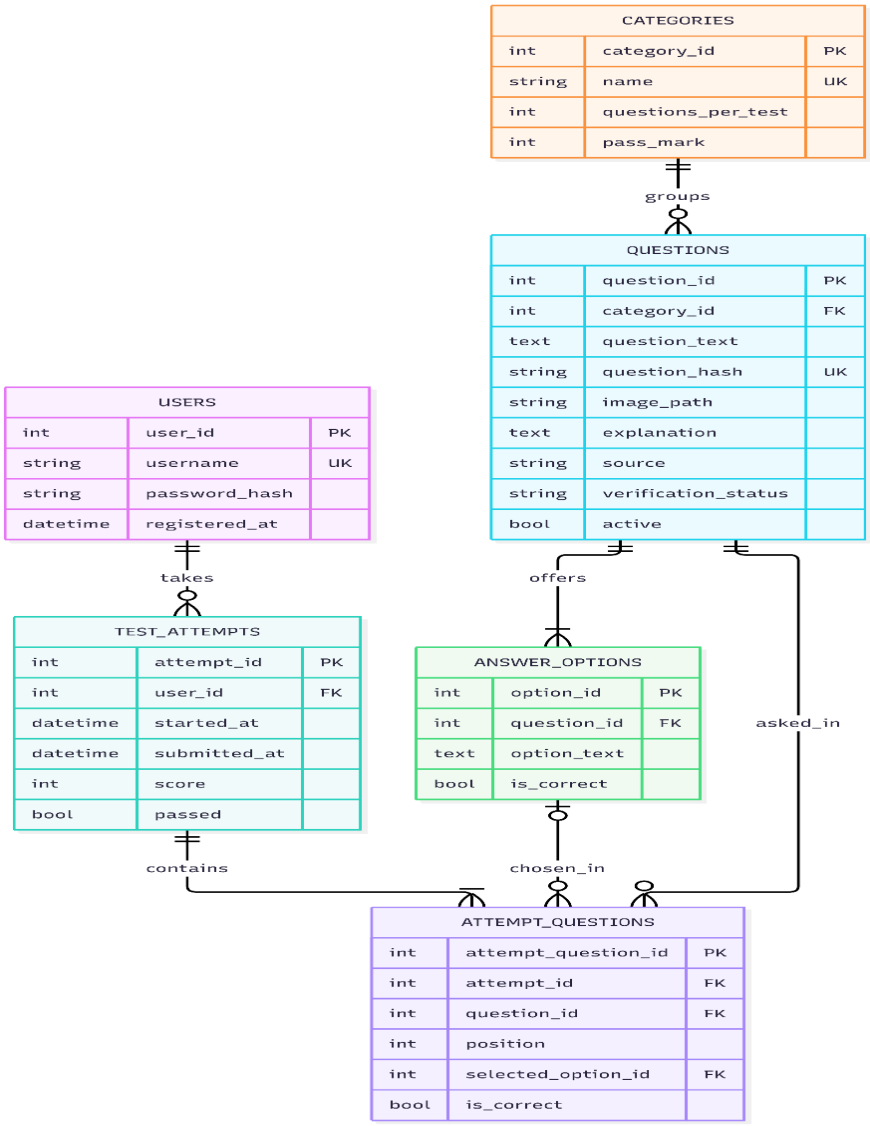
We use Google Docs to post our side of the work. Each member works in their allocated section. Once everyone has posted their work, we will combine the sections in the main System Design document for 30 October. We will then check that the sections follow the same project decisions and that the headings and formatting are consistent. After checking the complete document, we will save it as a PDF for submission.

GitHub is where the members doing development share their code. If we agree on a change in WhatsApp that affects the application, we will also record it in the relevant documentation or repository. This will help us keep the report up to date with what has actually been developed.

4.5 Database Design

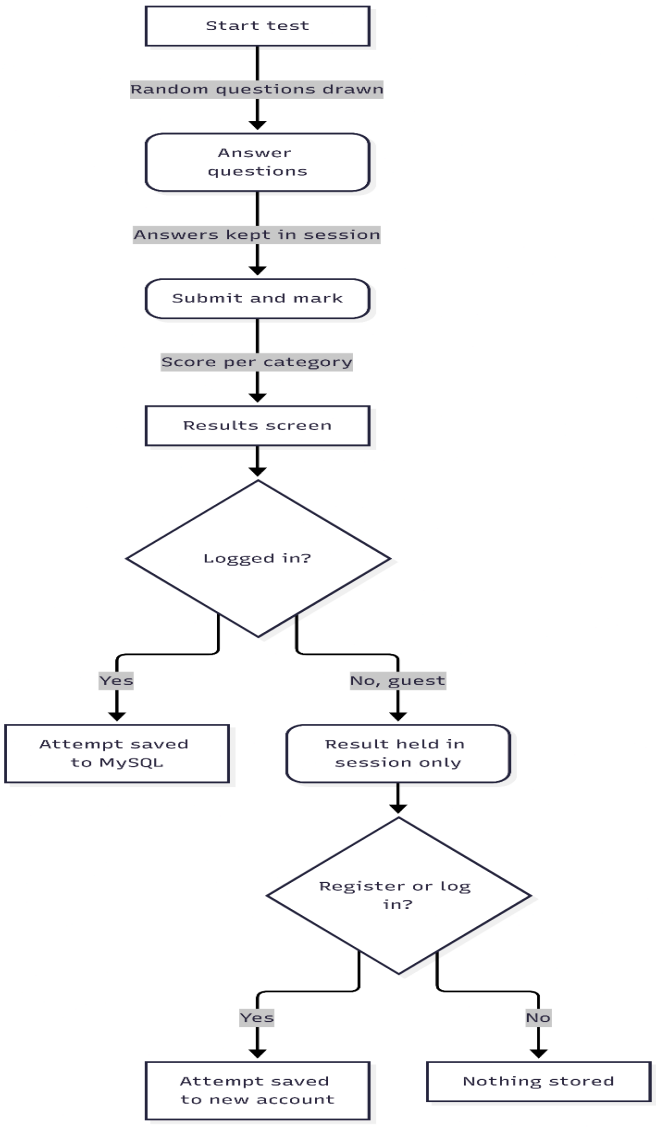
The database is a MySQL database made up of six tables (Users, Categories, Questions, Answer Options, Test Attempts and Attempt Questions) that are normalised to third normal form, so every fact is stored in one place only. Question wording, categories and sources live with the question, and a learner's marks for each category are worked out from their saved answers when needed, not stored a second time and left to go out of date. The most important quality is that it protects the accuracy of the question bank, which the whole project depends on. The database itself rejects duplicate questions and duplicate options, refuses a second correct answer for a question, and will not accept an answer that belongs to a different question. A question also cannot be marked as verified until it has at least two options and exactly one correct one. Once a question has appeared in a test, its wording is locked and it is retired and replaced with a corrected version instead, so a learner's old results always show what they were actually asked. Because question selection and marking only read data that has been checked in this way, the application code does not need to repeat those checks. The database is also small and quick enough for a free hosting plan, since indexes match the queries the backend runs most often. It keeps little personal data (a username and a hashed password), and the application connects with an account that can only read data and add new test records.

Please see the ERD below:


CATEGORIES
int category_id PK
string name UK
int questions_per_test
int pass_mark
groups
QUESTIONS
int question_id PK
int category_id FK
text question_text
string question_hash UK
string image_path
text explanation
string source
string verification_status
bool active
USERS
int user_id PK
string username UK
string password_hash
datetime registered_at
takes
offers
TEST_ATTEMPTS
int attempt_id PK
int user_id FK
datetime started_at
datetime submitted_at
int score
bool passed
ANSWER_OPTIONS
int option_id PK
int question_id FK
text option_text
bool is_correct
asked_in
contains
chosen_in
ATTEMPT_QUESTIONS
int attempt_question_id PK
int attempt_id FK
int question_id FK
int position
int selected_option_id FK
bool is_correct

Simulated user flow based off database design

Using ERD we can establish a simulated flow as to how a user experiences booting up the app.



Proposed SQL for our Database

```
-- ================================================================
-- CLLT Mock Test Application: database script (MySQL 8.0.16 or later)
-- Group 33, Richfield IT Project 700
-- Follows the Data Dictionary: six tables, 3NF, IDs as primary keys.
-- Additions beyond the dictionary are marked "ADDITION" in comments.
-- ================================================================

CREATE DATABASE IF NOT EXISTS clt_mock_test
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE clt_mock_test;

-- ----------------------------------------------------------------
-- 1. Users
-- ----------------------------------------------------------------
CREATE TABLE users (
    user_id          INT UNSIGNED NOT NULL AUTO_INCREMENT,
    username         VARCHAR(50)  NOT NULL,
    password_hash    VARCHAR(255) NOT NULL,   -- salted hash only, never the real password
    date_registered  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id),
    UNIQUE KEY uq_users_username (username),
    CONSTRAINT chk_users_username_length CHECK (CHAR_LENGTH(username) >= 3)
) ENGINE = InnoDB;

-- ----------------------------------------------------------------
-- 2. Categories
-- ADDITION: questions_per_test and pass_mark. The dictionary says a pass
-- must take the required mark for each category into account, so those
-- numbers need a home. Both default to 0 until the official figures are
-- confirmed. Remove the two columns if the group keeps them in Java config.
-- ----------------------------------------------------------------
CREATE TABLE categories (
    category_id         INT UNSIGNED      NOT NULL AUTO_INCREMENT,
    category_name       VARCHAR(60)       NOT NULL,
    questions_per_test  SMALLINT UNSIGNED NOT NULL DEFAULT 0,
    pass_mark           SMALLINT UNSIGNED NOT NULL DEFAULT 0,
    PRIMARY KEY (category_id),
    UNIQUE KEY uq_categories_name (category_name),
    CONSTRAINT chk_categories_pass_mark CHECK (pass_mark <= questions_per_test)
```

```
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- 3. Questions
-- ADDITION: image_key, a generated column used only so that the duplicate
-- check works for sign questions that share the same wording but have
-- different images. Never insert into it.
-- ---------------------------------------------------------------------
CREATE TABLE questions (
    question_id         INT UNSIGNED NOT NULL AUTO_INCREMENT,
    category_id         INT UNSIGNED NOT NULL,
    question_text       VARCHAR(500) NOT NULL,
    image_path          VARCHAR(200) NULL,
    explanation         TEXT         NULL,
    source              VARCHAR(255) NOT NULL,
    verification_status  ENUM('Unverified', 'Verified', 'Rejected')
                             NOT NULL DEFAULT 'Unverified',
    is_active           BOOLEAN      NOT NULL DEFAULT TRUE,
    image_key           VARCHAR(200) GENERATED ALWAYS AS (COALESCE(image_path, ''))
STORED,
    PRIMARY KEY (question_id),
    UNIQUE KEY uq_questions_text_image (question_text, image_key),
    KEY idx_questions_selection (category_id, is_active, verification_status),
    CONSTRAINT fk_questions_category
        FOREIGN KEY (category_id) REFERENCES categories (category_id)
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- 4. Answer Options
-- correct_for_question is a generated column: it holds the question ID on
-- the correct option and NULL on the others. The unique key on it means a
-- question can never have two correct options.
-- (MySQL rule: the foreign key on question_id must keep the default
-- RESTRICT actions because question_id feeds a stored generated column.)
-- ---------------------------------------------------------------------
CREATE TABLE answer_options (
    option_id           INT UNSIGNED NOT NULL AUTO_INCREMENT,
    question_id          INT UNSIGNED NOT NULL,
    option_text         VARCHAR(255) NOT NULL,
    is_correct          BOOLEAN      NOT NULL DEFAULT FALSE,
    correct_for_question  INT UNSIGNED GENERATED ALWAYS AS (IF(is_correct, question_id,
NULL)) STORED,
    PRIMARY KEY (option_id),
    UNIQUE KEY uq_options_question_option (question_id, option_id),
```

```
    UNIQUE KEY uq_options_one_correct (correct_for_question),
    UNIQUE KEY uq_options_text_per_question (question_id, option_text),
    CONSTRAINT fk_options_question
        FOREIGN KEY (question_id) REFERENCES questions (question_id)
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- 5. Test Attempts
-- user_id is empty for a guest, as the dictionary specifies. submitted_at,
-- score and passed stay empty until the attempt is marked.
-- ---------------------------------------------------------------------
CREATE TABLE test_attempts (
    attempt_id    INT UNSIGNED      NOT NULL AUTO_INCREMENT,
    user_id       INT UNSIGNED      NULL,
    started_at    DATETIME          NOT NULL,
    submitted_at  DATETIME          NULL,
    is_completed  BOOLEAN           NOT NULL DEFAULT FALSE,
    score         SMALLINT UNSIGNED NULL,
    passed        BOOLEAN           NULL,
    PRIMARY KEY (attempt_id),
    KEY idx_attempts_user_submitted (user_id, submitted_at),
    CONSTRAINT fk_attempts_user
        FOREIGN KEY (user_id) REFERENCES users (user_id),
    CONSTRAINT chk_attempts_dates
        CHECK (submitted_at IS NULL OR submitted_at >= started_at),
    CONSTRAINT chk_attempts_completed
        CHECK (is_completed = FALSE
            OR (submitted_at IS NOT NULL AND score IS NOT NULL AND passed IS NOT
NULL))
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- 6. Attempt Questions
-- The composite foreign key (question_id, selected_option_id) forces the
-- chosen option to belong to the same question. A NULL selected_option_id
-- means the question was not answered, and MySQL skips the check for it.
-- ---------------------------------------------------------------------
CREATE TABLE attempt_questions (
    attempt_question_id  INT UNSIGNED      NOT NULL AUTO_INCREMENT,
    attempt_id           INT UNSIGNED      NOT NULL,
    question_id          INT UNSIGNED      NOT NULL,
    question_order       SMALLINT UNSIGNED NOT NULL,
    selected_option_id   INT UNSIGNED      NULL,
    PRIMARY KEY (attempt_question_id),
```

```
  UNIQUE KEY uq_aq_attempt_question (attempt_id, question_id),
  UNIQUE KEY uq_aq_attempt_order (attempt_id, question_order),
  KEY idx_aq_question_option (question_id, selected_option_id),
  CONSTRAINT fk_aq_attempt
    FOREIGN KEY (attempt_id) REFERENCES test_attempts (attempt_id)
    ON DELETE CASCADE,
  CONSTRAINT fk_aq_question
    FOREIGN KEY (question_id) REFERENCES questions (question_id),
  CONSTRAINT fk_aq_selected_option
    FOREIGN KEY (question_id, selected_option_id)
    REFERENCES answer_options (question_id, option_id),
  CONSTRAINT chk_aq_order CHECK (question_order >= 1)
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- Triggers: rules a plain constraint cannot express
-- ---------------------------------------------------------------------
DELIMITER $$

-- New questions must start as Unverified (their options do not exist yet).
CREATE TRIGGER trg_questions_bi
BEFORE INSERT ON questions
FOR EACH ROW
BEGIN
  IF NEW.verification_status = 'Verified' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Insert new questions as Unverified, add the answer options,
then verify.';
  END IF;
END$$

CREATE TRIGGER trg_questions_bu
BEFORE UPDATE ON questions
FOR EACH ROW
BEGIN
  -- A question can only be Verified with 2+ options and exactly one correct.
  IF NEW.verification_status = 'Verified' THEN
    IF (SELECT COUNT(*) FROM answer_options
        WHERE question_id = NEW.question_id) < 2
       OR (SELECT COUNT(*) FROM answer_options
           WHERE question_id = NEW.question_id AND is_correct = TRUE) <> 1 THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'A verified question needs at least two options and exactly
one correct option.';
```

```
      END IF;
   END IF;

   -- Keep old results accurate: a question that has been used is never
   -- reworded. Retire it (is_active = FALSE) and add a corrected version.
   IF (NEW.question_text <> OLD.question_text
       OR NEW.category_id <> OLD.category_id
       OR NOT (NEW.image_path <=> OLD.image_path))
      AND EXISTS (SELECT 1 FROM attempt_questions
                  WHERE question_id = OLD.question_id) THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'This question has been used in a test. Set it inactive and add
a corrected version instead.';
   END IF;
END$$

-- Same protection for the options of a question that has been used.
CREATE TRIGGER trg_answer_options_bu
BEFORE UPDATE ON answer_options
FOR EACH ROW
BEGIN
   IF (NEW.option_text <> OLD.option_text
       OR NEW.is_correct <> OLD.is_correct
       OR NEW.question_id <> OLD.question_id)
      AND EXISTS (SELECT 1 FROM attempt_questions
                  WHERE question_id = OLD.question_id) THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'This question has been used in a test. Set it inactive and add
a corrected version instead.';
   END IF;
END$$

DELIMITER ;

-- ---------------------------------------------------------------------
-- Views
-- ---------------------------------------------------------------------

-- Questions the backend may put in a new test: Verified, active, and still
-- holding 2+ options with exactly one correct (guards against later deletes).
CREATE VIEW v_test_ready_questions AS
SELECT q.question_id,
       q.category_id,
       q.question_text,
```

```
       q.image_path,
       q.explanation
FROM questions q
WHERE q.verification_status = 'Verified'
  AND q.is_active = TRUE
  AND (SELECT COUNT(*) FROM answer_options o
        WHERE o.question_id = q.question_id) >= 2
  AND (SELECT COUNT(*) FROM answer_options o
        WHERE o.question_id = q.question_id AND o.is_correct = TRUE) = 1;

-- Marks for each category, worked out when read, never stored (3NF).
CREATE VIEW v_attempt_category_results AS
SELECT aq.attempt_id,
       c.category_id,
       c.category_name,
       COUNT(*)                          AS questions_asked,
       COALESCE(SUM(o.is_correct), 0)        AS correct_answers,
       c.pass_mark                        AS required_mark,
       (COALESCE(SUM(o.is_correct), 0) >= c.pass_mark) AS category_passed
FROM attempt_questions aq
JOIN questions  q ON q.question_id = aq.question_id
JOIN categories c ON c.category_id = q.category_id
LEFT JOIN answer_options o ON o.option_id = aq.selected_option_id
GROUP BY aq.attempt_id, c.category_id, c.category_name, c.pass_mark;

-- ---------------------------------------------------------------------
-- Starting data
-- Set questions_per_test and pass_mark once the official figures are
-- confirmed, for example:
--   UPDATE categories SET questions_per_test = ?, pass_mark = ?
--    WHERE category_name = 'Road signs';
-- ---------------------------------------------------------------------
INSERT INTO categories (category_name) VALUES
   ('Rules of the road'),
   ('Road signs'),
   ('Controls of the vehicle');

-- ---------------------------------------------------------------------
-- Application account (least privilege). The backend reads everything,
-- and only inserts into the three tables learners generate data in.
-- Question bank changes are made by the team with an admin account.
-- Replace the password before use.
-- ---------------------------------------------------------------------
```

```
CREATE USER IF NOT EXISTS 'clt_app'@'localhost' IDENTIFIED BY
'CHANGE_ME_BEFORE_USE';
GRANT SELECT ON clt_mock_test.* TO 'clt_app'@'localhost';
GRANT INSERT ON clt_mock_test.users             TO 'clt_app'@'localhost';
GRANT INSERT ON clt_mock_test.test_attempts     TO 'clt_app'@'localhost';
GRANT INSERT ON clt_mock_test.attempt_questions TO 'clt_app'@'localhost';

-- ---------------------------------------------------------------------
-- How a question is added (run by the team, not the app)
-- ---------------------------------------------------------------------
-- 1. INSERT INTO questions (category_id, question_text, source)
--    VALUES (?, 'Question wording', 'Where it came from');
-- 2. INSERT INTO answer_options (question_id, option_text, is_correct)
--    VALUES (LAST_INSERT_ID(), ...), one row per option, one marked TRUE.
-- 3. UPDATE questions SET verification_status = 'Verified'
--    WHERE question_id = ?;   -- fails unless the options are right.

-- ---------------------------------------------------------------------
-- Queries the backend will use (? = JDBC parameter)
-- ---------------------------------------------------------------------
-- Pick random questions for one category:
--   SELECT question_id FROM v_test_ready_questions
--    WHERE category_id = ? ORDER BY RAND() LIMIT ?;
-- Load the options for the chosen questions in one query:
--   SELECT option_id, question_id, option_text, is_correct
--     FROM answer_options WHERE question_id IN (?, ?, ...);
-- Category breakdown for a saved attempt:
--   SELECT * FROM v_attempt_category_results WHERE attempt_id = ?;
```

4.6 Program Design (Program Pseudo code)

This section describes the framework and project structure of the backend, followed by the pseudocode for the bulk question import added to the backend in this phase.

Framework and technologies

The backend is a Java 17 application built with Spring Boot 4.1.2 and managed with Apache Maven (Apache Software Foundation).

This repository includes the Maven wrapper, so that every team member builds the project with the same Maven version.

Spring Web MVC provides the controllers that expose a REST style interface to the HTML pages. Spring Data JPA maps the model classes to database tables and supplies the repository implementations, which removes the need for hand written SQL in the data layer.

Sessions, handled with HttpSession, identify the logged in users and tie guest test attempts to the browser session that started them.

Data is stored in an H2 database (H2 Database Engine,) kept in a file, as set in application.properties, so questions and results survive a restart. H2 was chosen for development because it needs no separate database server. The project proposal names MySQL as the target database. Because the data layer uses JPA, moving to MySQL should only need a MySQL driver dependency and new datasource properties. The frontend pages in the Views folder are plain HTML, CSS and JavaScript, as set out in the proposal, and use the backend endpoints.

```
spring.application.name=backend
spring.datasource.url=jdbc:h2:file:./data/mocktest
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=true
app.admin-key=change-me
server.port=8080
```

Figure 4.6.1: Database and admin key settings in application.properties

Project structure

The project is stored in one Git repository, with the backend and the frontend in separate top level folders.

The backend follows the layered structure described in the proposal, in which each layer has one job.

Entities in the model package represent the stored data, repositories in the repo package read and write it, services hold the business logic, and controllers receive requests and return responses.

Data transfer objects in the dto package define the shape of the requests and responses. The Views folder holds the three pages of the test flow, with one stylesheet for each page, and forms the view layer.

The config package holds CorsConfig and DataSeeder, the service package also holds a PasswordHasher helper class, and the test folder holds the default BackendApplicationTests class.

Figures 4.6.2 to 4.6.8 show the project as it appears in the IDE.

```
Learners-License-Mock-Test
  Backend
    .mvn
    data                    (H2 database file)
    src
```

```
        main
            java/com/mocktest
                BackendApplication.java
                config
                    CorsConfig.java, DataSeeder.java
                controller
                    AdminController.java, AuthController.java,
                    TestController.java
                dto
                    Dto.java
                model
                    AnswerOption.java, AttemptQuestion.java,
                    Category.java, Question.java,
                    TestAttempt.java, User.java
                repo
                    AnswerOptionRepository.java,
                    AttemptQuestionRepository.java,
                    CategoryRepository.java, QuestionRepository.java,
                    TestAttemptRepository.java, UserRepository.java
                service
                    PasswordHasher.java, QuestionService.java,
                    TestService.java
            resources
                application.properties
        test/java/com/mocktest
            BackendApplicationTests.java
    target                      (build output)
    .gitattributes, .gitignore
    mvnw, mvnw.cmd
    pom.xml
Views
    start-test.html, test-question.html, results.html
    css
        start-test.css, test-question.css, results.css
README.md
```

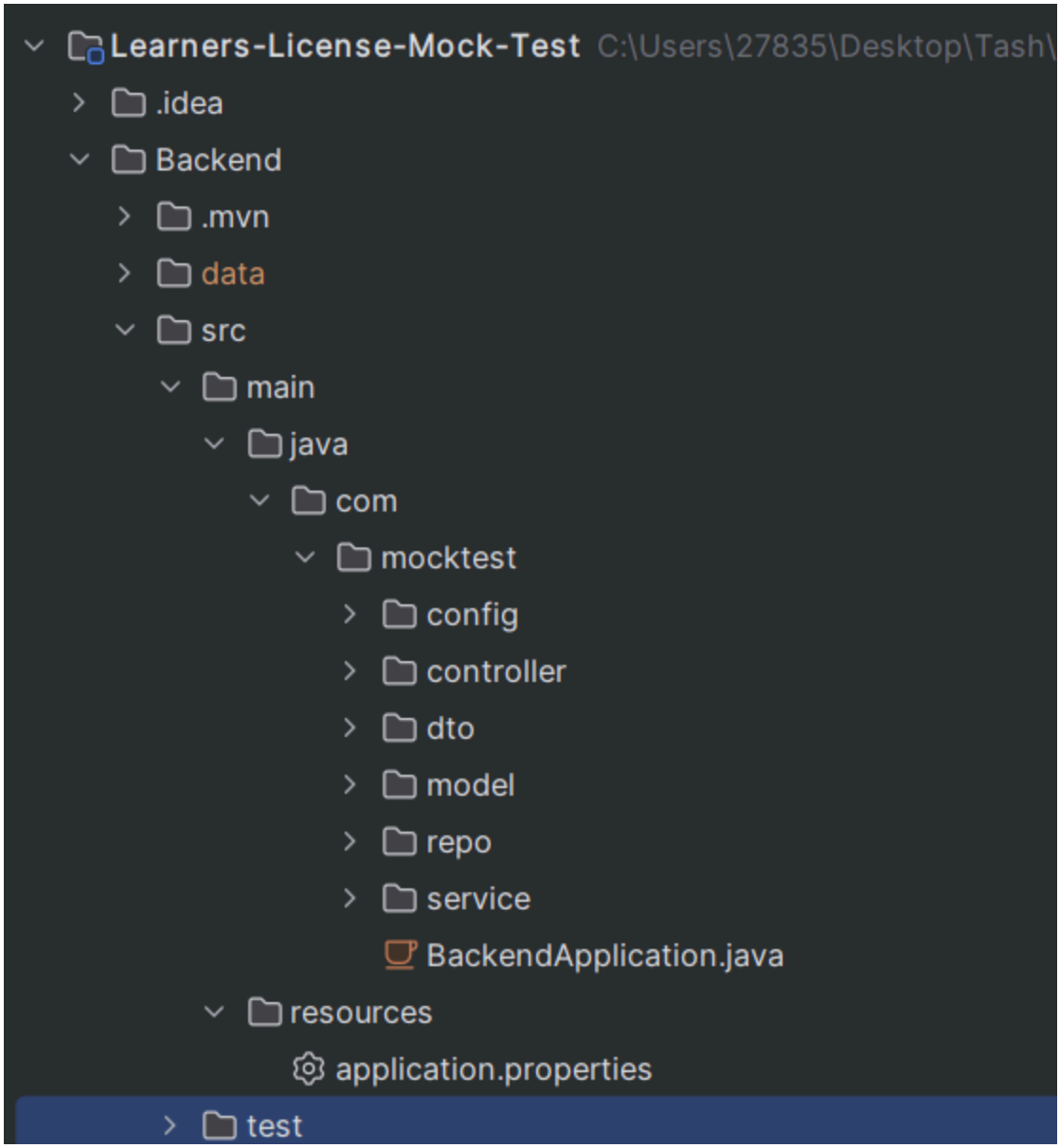


Figure 4.6.2: Backend folder structure in the IDE

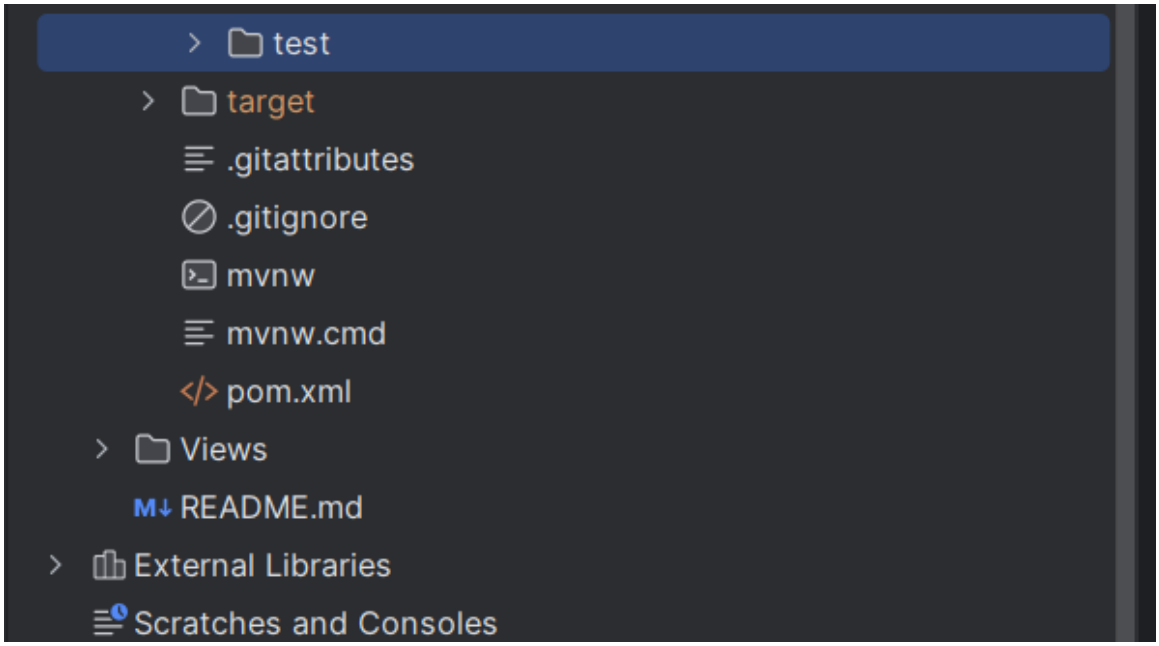


Figure 4.6.3: Build files, Views folder and README in the IDE

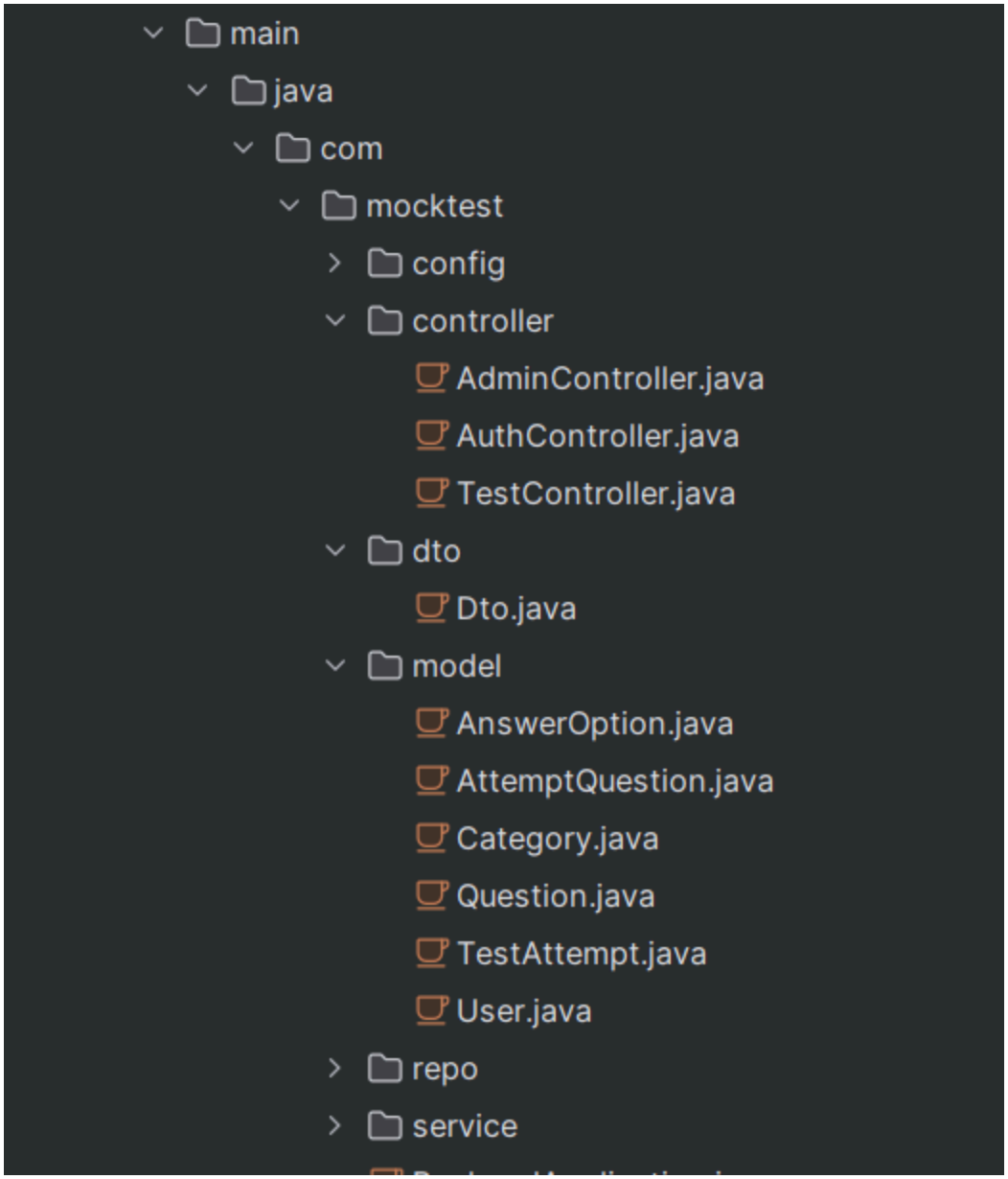


Figure 4.6.4: Controller, dto and model packages in the IDE

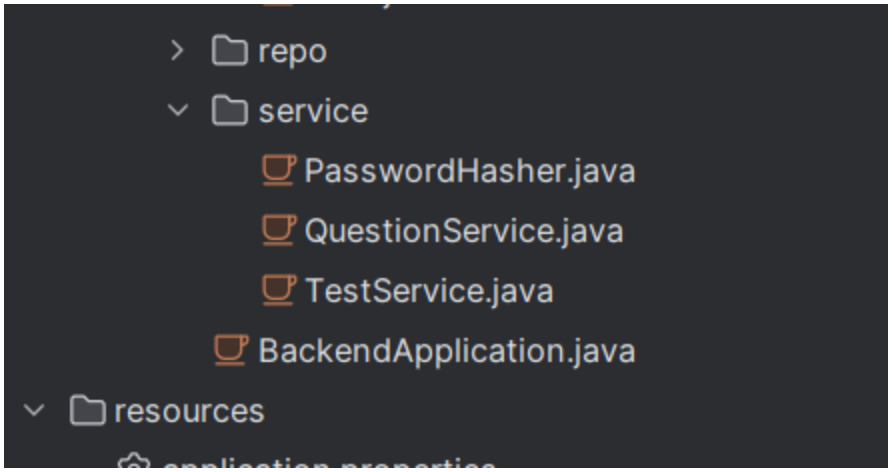


Figure 4.6.5: Service package in the IDE

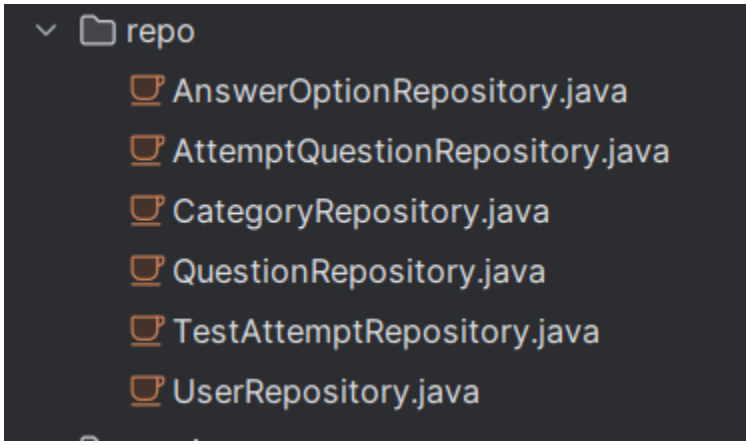


Figure 4.6.6: Repo package in the IDE

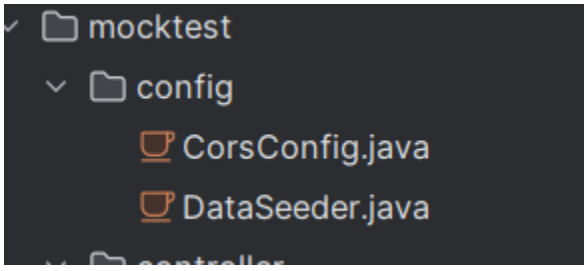


Figure 4.6.7: Config package in the IDE

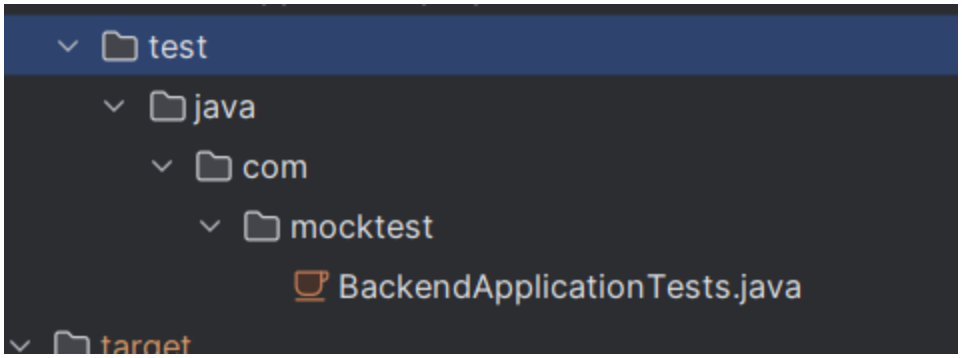


Figure 4.6.8: Test folder in the IDE

4.6.3 Admin key check

All admin endpoints are protected by a shared key. The caller sends the key in the X-Admin-Key header and the backend compares it with the app.admin-key property in application.properties. The bulk endpoint calls this existing check before doing anything else. The key is currently a placeholder value and must be changed before the system is deployed.

```
FUNCTION check(key)
  IF key is empty OR key is not equal to the stored admin key THEN
    REJECT the request with 403 Forbidden
  END IF
END FUNCTION
```

```java
@RestController
@RequestMapping("/api/admin/questions")
public class AdminController {
    private final QuestionService service;
    private final String adminKey;

    public AdminController(QuestionService service, @Value("${app.admin-key}") String adminKey) {
        this.service = service; this.adminKey = adminKey;
    }

    private void check(String key) {
        if (key == null || !key.equals(adminKey))
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Admin key required (X-Admin-Key header)");
    }

    @GetMapping
    public List<Dto.AdminQuestion> list(@RequestHeader(value = "X-Admin-Key", required = false) String key) {
        check(key);
        return service.list();
    }
```

Figure 4.6.9: Start of AdminController, showing the base path, constructor and check method

```java
private void check(String key) {
    if (key == null || !key.equals(adminKey))
        throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Admin key required (X-Admin-Key header)");
}
```

Figure 4.6.10: Admin key check in AdminController

4.6.4 Bulk question import

The proposal commits to a question bank of close to 1200 verified NaTIS questions, but the existing admin endpoint could only add one question per request.

A bulk endpoint was therefore added to AdminController.

It reuses the existing QuestionService, so the validation and storage rules already written by the team apply to every imported question.

The endpoint is POST /api/admin/questions/bulk.

It takes the admin key header, an optional verify parameter that defaults to false, and a list of question requests in the request body.

Each question is processed on its own, so one invalid question does not stop the rest of the import.

Any failure is recorded with the position of the question in the list so that it can be corrected and resubmitted. Imported questions are saved as Unverified by default, which matches the project requirement that only verified NaTIS questions are used in tests.

Setting verify to true marks the imported questions as Verified straight away, which is intended for testing only.

```
FUNCTION bulkAdd(key, verify, requests)
  check(key)
  SET created TO an empty list
  SET failed TO an empty list

  FOR i FROM 0 TO (size of requests - 1)
    TRY
      SET id TO QuestionService.create(requests[i]).questionId
      IF verify is true THEN
        QuestionService.setStatus(id, "Verified")
      END IF
      ADD id TO created
    CATCH error
      ADD (index i, error message) TO failed
    END TRY
  END FOR

  RETURN imported = size of created, failed = failed, questionIds = created
END FUNCTION
```

```java
@PostMapping("/bulk")
public Map<String, Object> bulkAdd(@RequestHeader(value = "X-Admin-Key", required = false) String key,
                                   @RequestParam(defaultValue = "false") boolean verify,
                                   @RequestBody List<Dto.QuestionRequest> requests) {
    check(key);
    List<Long> created = new ArrayList<>();
    List<Map<String, Object>> failed = new ArrayList<>();
    for (int i = 0; i < requests.size(); i++) {
        try {
            Long id = service.create(requests.get(i), null).questionId;
            if (verify) service.setStatus(id, "Verified");
            created.add(id);
        } catch (Exception e) {
            failed.add(Map.of("index", i, "error", String.valueOf(e.getMessage())));
        }
    }
    return Map.of("imported", created.size(), "failed", failed, "questionIds", created);
}
```

Figure 4.6.11: bulkAdd method in AdminController

4.6.5 Input and output data

Each item in the request list uses the existing QuestionRequest structure, which holds the category, question text, image path, explanation, source and a list of answer options.

Each option has its text and a flag showing whether it is the correct answer.

The response reports how many questions were imported, which ones failed and why, and the identifiers of the questions that were created.

```
INPUT (list of questions)
   categoryId, questionText, imagePath, explanation, source
   options: list of (text, correct)

OUTPUT
   imported: number of questions saved
   failed: list of (index, error)
   questionIds: list of new question identifiers
```

4.7 Interface Design

The Interface design for the web-based NaTIS Computerised Learner’s Licence Test (CLLT) mock application, we want to focus on user-friendliness, accessibility, inclusive and universal Design.
This will be achieved with a clean look, that while up to date with current trends, allows maximum visual feedback to users. We will use contrast and colours to emphasise important UI elements to direct the users attention to. Clear navigational UI will ensure users do not feel lost in the web-app.
This will ensure that the largest demographic possible will find this app easy to use.

The application's front-end architecture is organized into distinct functional views to streamline navigation and maintain a low barrier to entry:

Login and Registration Pages
This must be designed with minimal friction, allowing users to quickly access their accounts or proceed as guests.

It will feature clear input validation for username and password fields, secure password hashing handling behind the scenes, and distinct visual prompts for form errors.

Test Questions Interface (The Mock Test Arena)
This will be built to mirror the core elements of the official NaTIS CLLT testing environment. Displays the active question text alongside randomised multiple-choice options (A, B, C). Incorporates persistent trackers such as a live countdown timer, question progress indicators (e.g., Question 12 of 30), and intuitive "Next" or "Previous" navigation controls.

We want to design this to be accessible for both mobile and desktop platforms from the start.

Results Main Dashboard
This will be loaded immediately upon test submission, presenting a performance summary. It will clearly display if the candidate passed or failed overall, alongside their total percentage mark and completion time.

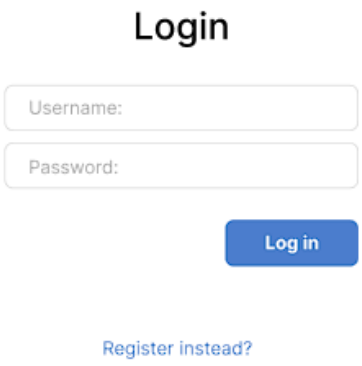
Results Section-Specific Breakdown
This directly addresses the diagnostic shortcomings of existing commercial apps.
It provides a granular breakdown mapping performance across the official NaTIS categories: **Road Rules, Road Signs, and Vehicle Controls**.

This allows us to highlight specific incorrect answers and weak topics, allowing candidates to review mistakes and understand the correct traffic regulations before attempting another

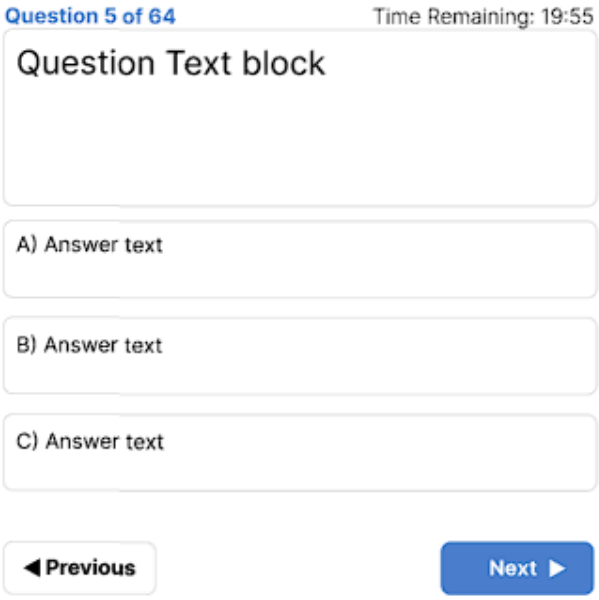
Attached Resources:

Attached are the Figma mockups for each of these corresponding pages:

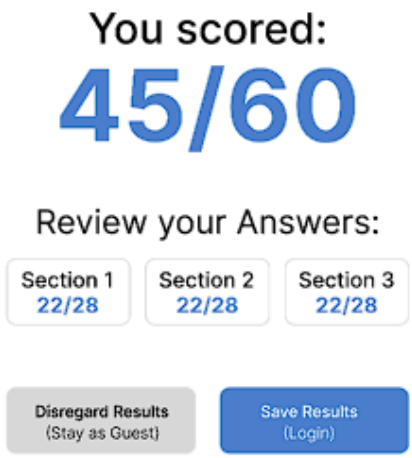
Login:



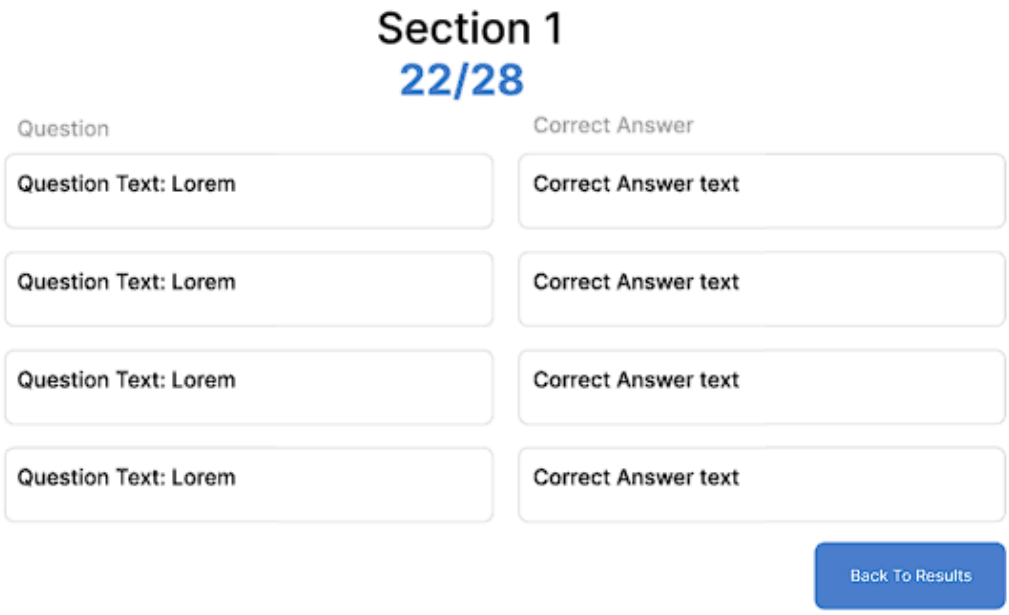
Test Question Interface:



Main Results Dashboard:



Results, Section Specific:



4.8 Security and Backup Design

We need to look after the information that learners give us when they use the application. This includes their login details, answers and test results. We also need to protect the question bank because a change to a question or correct answer could affect the results learners receive.

The backup side is how we will recover our work if something gets deleted or the database stops working. We do not want to lose the questions we collected or the attempts learners have completed. The measures below follow Sections 3.4.5 and 3.7 and will need to be checked during development and testing.

4.8.1 Password Protection and Login Sessions

When a learner registers, we will store their password as a salted hash. We will not keep a readable copy of the password in the database. The application will use the stored hash to check whether the password entered during login is correct.

Each password must have its own randomly generated salt. This helps protect accounts even if two learners happen to choose the same password. The backend documentation currently describes PBKDF2 in the PasswordHasher component. We need to check that it creates a unique salt and uses a suitable work factor, following OWASP's guidance (OWASP, n.d.-a). The work factor determines how much processing is needed to check a password.

We must also make sure that passwords and their hashes are excluded from application logs and responses sent to the frontend.

After login, the backend will use a session to keep track of the learner while they move between pages. The session identifier will change when they log in, and logging out will end the session. We will set an inactivity timeout as well. For instance, someone might use a shared computer and forget to log out. Their account should not stay accessible for the next person.

The hosted application will use Secure and HttpOnly session cookies with an appropriate SameSite setting. These settings control how the browser sends and accesses the session cookie (OWASP, n.d.-b).

Guest users will still be able to take a test without registering. Their attempt will have no user ID and will belong to the browser session that started it. A page refresh should keep the current attempt available, as required in Phase 3. Another session must not be able to open that guest's test.

4.8.2 Access to Results and Question Management

Each registered learner must only have access to their own attempts and results. We will check ownership in the backend when they open an attempt, save an answer or submit the test.

For instance, a learner could try changing the attempt ID in a request to see another person's results. The backend must refuse that request if the attempt does not belong to them. We cannot rely on keeping the link hidden because someone could send the request without using the page.

These permission checks must happen whenever protected information is requested, as recommended by OWASP (n.d.-c).

Only authorised group members will be allowed to manage the question bank. This covers adding questions, checking their sources, changing their verification status and retiring questions that should no longer be used.

The program-design documentation currently describes an administrative key sent in the X-Admin-Key header. The placeholder key must be changed before deployment. Its replacement will be a strong secret stored outside the source code repository.

We must keep the key out of public JavaScript files, screenshots and logs. If it is exposed, we will replace it. The shared key is a basic arrangement for our prototype. If we continue developing the application, separate administrator accounts would make it easier to control access and record who changed a question.

4.8.3 Checking Inputs and Test Results

We will check the information received by the backend before it is processed or saved. This includes checking required fields, input lengths and whether the records being requested exist.

The interface will help learners correct their inputs, but the backend still needs to do its own checks. Someone could send a request directly and bypass the checks on the page.

When an answer is received, the backend must confirm that the option belongs to the question and that the question belongs to that learner's attempt. This will prevent an answer from being saved against the wrong test.

In Phase 3, we specified parameterised queries to protect against SQL injection. The current backend documentation describes JPA repositories for database access. We will need to check that input values are bound safely and that any custom queries follow the same rule. User input must not be joined into SQL commands.

Question text and explanations will be displayed as text. This avoids treating the content as executable HTML. We will also need protection against cross-site request forgery for requests that change data using login cookies. This is where another website tries to make the browser send a request that the learner did not intend to make.

Java will calculate the marks from the answers stored in the database. The backend will not accept a score or pass result supplied by the browser. Questions sent during an active test must also exclude the flags that identify the correct options.

We will only select active, verified questions for new tests. If we find an error in a question that has already been used, we will keep that version and make it inactive. A corrected version can then be added for future tests. Previous attempts will still refer to the question the learner actually answered.

4.8.4 Security of the Hosted Application

The hosted application will use HTTPS for communication between the browser and server. This will protect login details, session information and answers while they are being sent.

Database connection details will stay in the server configuration. The backend will use a database account with only the permissions needed for the application, and access to the database will be restricted.

Before deployment, we will check the tools and settings used during development. Database consoles and detailed error pages must be disabled or restricted. A learner should receive an understandable error message if something fails, without being shown internal settings or database details.

We will use logs to investigate errors and failed access attempts. The logs must exclude passwords, administrative keys and session identifiers. Repeated login attempts will also be limited to make automated password guessing more difficult.

If the frontend and backend run on separate addresses during development, we will configure the backend to allow the required frontend addresses. Requests from those addresses must still pass the usual permission and ownership checks.

4.8.5 Backup Schedule and Storage

We need to back up the database and question images separately from our code history. GitHub keeps the changes made to the code, but it does not automatically keep a copy of the learner records in the running database.

We propose a daily backup of MySQL while the application is in use. We will also make a backup before importing a large batch of questions, changing the database or doing a major deployment update. If the change causes a problem, we can return to the backup taken beforehand.

Our proposed schedule is to keep seven daily backups and four weekly backups. They will be encrypted and stored separately from the application. Only the members responsible for maintenance will have access. We must not upload backups containing account information to a public repository or post them in the WhatsApp group.

The question images must be included with the database backup. For instance, restoring a question record will not help much if its road-sign image is missing. Keeping the images and database from the same backup point will help avoid that problem.

For H2 during local development, we will use a consistent backup procedure. One option is to stop the application cleanly before copying its database files. Copying the files while the application is changing them could leave us with an unreliable backup.

Members will regularly upload completed code to GitHub. Before demonstrations and submission, we will identify a working version. We will use Google Docs version history for the report and keep exported copies of the submitted phases.

4.8.6 Restoring Data and Checking Backups

If data is accidentally deleted or the database becomes damaged, we will stop further changes to the affected application. We will preserve a copy of the damaged data while the member responsible for database maintenance restores the latest suitable backup into a separate environment.

We will then check the restored application before putting it back into use. The expected records must be present, the relationships between tables must be correct and question images must still load.

We will also test the main learner functions. This includes registration, login, starting a test, saving answers, submitting an attempt and opening previous results. Question verification statuses must be checked so that unverified questions do not become available for testing after a restore.

A successful daily backup should limit data loss to no more than 24 hours. We still need to carry out a restore exercise and measure how long it takes before setting a recovery-time target.

We will nominate a group member to check that backups have completed. Another member will help verify a restored backup before a major demonstration or release. Keeping a backup file is only useful if we have checked that it can be restored.

Before public deployment, we will check the security measures as well. Passwords must be stored as hashes, and learners must be refused access to other users' attempts. Guest attempts must remain linked to their own sessions. We will also test that unauthorised question changes and invalid answer submissions are refused. These results and the backup restore results will be included in our testing evidence.